Bildklassifikation mit KI

Wie verbessere ich ein vorhandenes KI-Modell?

TSR · SGBlur · Panoramax
DE:310 und DE:311 mit Bildbeispielen





Prof. Dr. Raphael Volz · HSPF

Montag, 26. Oktober 2026 · 45 Minuten

Am Ende können Sie einen Modellversuch aufsetzen

01	**Verstehen**	Bekannte Bildbeispiele → Gewichte lernen → Klasse vorhersagen
02	**Vorbereiten**	Geprüfte Bildausschnitte (Bildausschnitte) fest aufteilen: Training, Validierung, Test
03	**Ausführen**	Die bereitgestellten Skripte starten und Berichte lesen
04	**Bewerten**	Ortstafeln erkennen; bisherige Klassen auf Rückschritte prüfen

Zwei Situationen: Box fehlt oder Klasse falsch

1 Box fehlt an der Ortstafel



Dobel: keine Box → kein Bildausschnitt
Detektion mit Vollbildern und Boxen verbessern

2 Box erkannt, Klasse falsch



Bernbach: Box vorhanden → bad
Mit geprüften Bildausschnitten nachtrainieren

Zuerst prüfen: Liegt der Fehler bei der Lokalisierung oder der Klassifikation?

Die Box stimmt – die neue Klasse fehlt



Realer Bildausschnitt · 212 × 148 Pixel

admin / Panoramax · CC BY-SA 4.0

Bernbach: Ortstafel lokalisiert

Historische Klassifikation: bad

DE-Modell: 134 Klassen
DE:310 und DE:311 fehlen

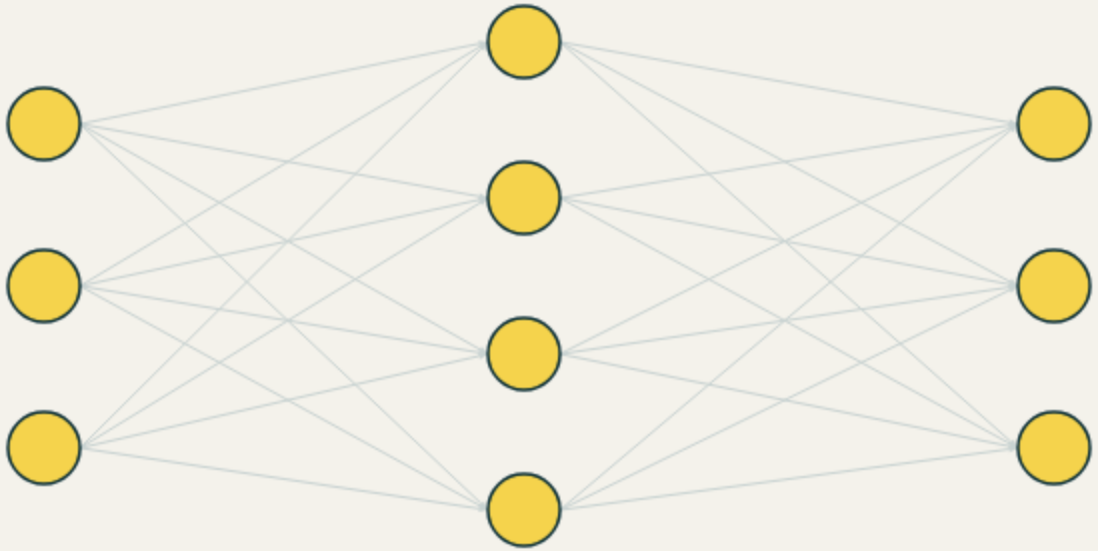
Trainingsbilder: Panoramax/classified_de_road_signs

Gewichte: Panoramax/classify_de_road_signs

Annotation in Panoramax verändert das Modell nicht, erst bei erneutem Lernen.

Ein neuronales Netz lernt Gewichte aus Beispielen

Überwachtes Lernen



$z = \Sigma\, w_i x_i + b \rightarrow$ Aktivierung

Pixelwerte **Gelernte Merkmale** **Klassenscores**

Eingabe: Zahlenwerte der Bildpixel

Gewichte + Aktivierungen erzeugen Klassenscores

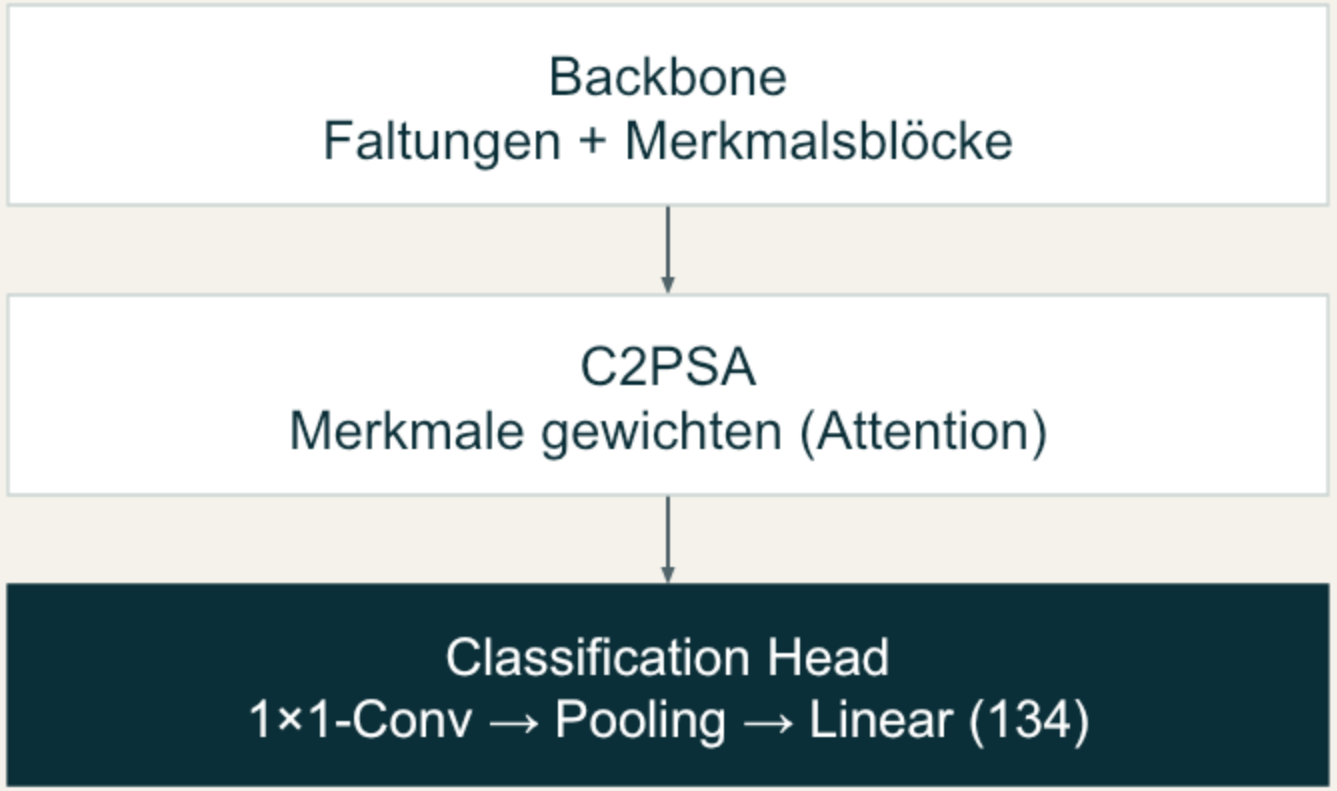
Loss misst den Unterschied (Fehler) zur bekannten Lösung des Beispiels

Backpropagation + Optimizer verändern die Gewichte

Training verändert Gewichte. Inferenz verwendet die gelernten Gewichte.

CNNs lernen lokale Merkmale im Bild

Im Projekt: YOLO26l-cls



Faltung: derselbe kleine Filter an vielen Bildpositionen

Backbone: lokale Muster zu Bildmerkmalen verbinden

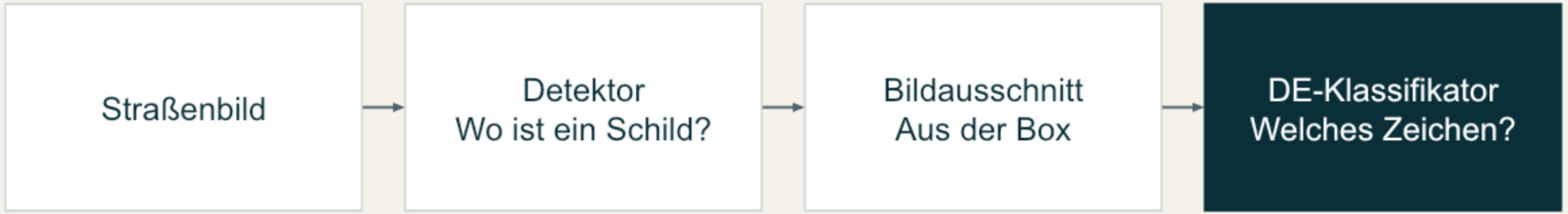
Attention: wichtige Merkmale unterschiedlich gewichten

Classification Head: Merkmale → 134 Klassenwerte

Architekturquelle: Jocher et al. (2026), YOLO26

Vorhandene Merkmale weiterverwenden; den Classification Head um zwei Klassen erweitern.

Zwei Lernaufgaben hinter einer Erkennung



Box fehlt → Detektor und Vollbildlabels prüfen

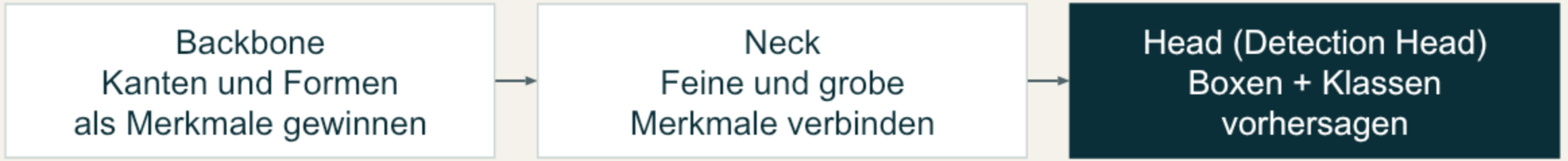
Box stimmt, Klasse falsch → Bildausschnitt, Klassen und Klassifikator prüfen

YOLO: eine Modellfamilie für verschiedene Aufgaben

YOLO = You Only Look Once

Variante	Eingabe	Ausgabe
Detektion	Ganzes Straßenbild	Boxen + Objektklassen + Scores
Klassifikation (−cls)	Ein Bildausschnitt	Ein Score je Zeichenklasse

Detektor



Klassifikator: Backbone → Classification Head → Klasse (ohne Neck)

Forschungsursprung: Redmon et al. (CVPR 2016), YOLO

Trainingsbilder und Modellgewichte

Datensatz: Bilder + Klassenlabels

Panoramax/classified_de_road_signs

Bildausschnitte von Verkehrszeichen

Bekannte Klasse je Beispiel

train.zip zum Lernen
val.zip für die Modellauswahl

Modell: gelernte Gewichte

Panoramax/classify_de_road_signs

YOLO26l-cls mit 134 Klassen

Sagt die Klasse eines Ausschnitts voraus

Fine-Tuning ergänzt DE:310 / DE:311

Für unseren Versuch bleiben alte Beispiele erhalten. Neue Ortstafeln kommen hinzu.

Vor der Kursaufteilung Dopplungen prüfen und zusammengehörende Bilder gruppieren.

Modelle: Gewichte, Dateien und App-Speicher

Modell	Parameter	Datei: MB .pt / iOS / Android	App-Puffer[1] Android
YOLO11n · Detektor 3 Klassen · 1024²	2.590.425	6 / 6 / 6	22 MB
YOLO26l-cls · DE 134 Klassen · 224²	13.006.278	26 / 26 / 26	1 MB

SGBlur: Modellquelle

HF: Panoramax/classify_de_road_signs

[1] Berechnete Puffer; Gewichte, Aktivierungen und GPU-Speicher kommen hinzu.

Historische Historische Gesamtapp-Beispiele: Android 646 MB PSS · iPhone 232 MB phys. Footprint.

Volle MB gerundet. Gesamtapp: unterschiedliche Last und Messgrößen; kein isolierter Modell-RAM.

YOLO26s im Dashcam-Vergleich

3 Fahrten · 31:47 Minuten · 65 feste Prüfbilder · 1024 × 1024

Detektor	Gefundene Boxen[1]	Fehlvorschläge[1]	Mac-CPU[2]
YOLO11n · aktuell	17 / 48	1	68 ms
YOLO26s · Kandidat	14 / 48	0	128 ms

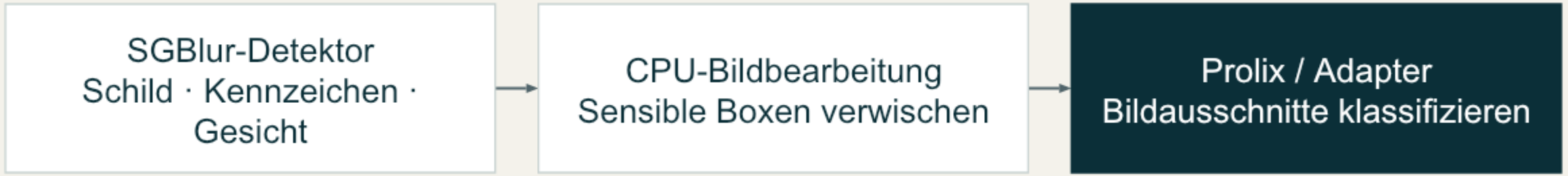
[1] Assistentenannotation: erkennbare Schildfronten ab 20 px · keine App-Fehlalarmrate

[2] Median über 3.814 Videobilder · One-to-many-Zweig · keine Telefonmessung

SGBlur: gepinnter YOLO26s-Checkpoint

Die Videoprüfung rechtfertigt noch keinen Modellwechsel in den Apps.

SGBlur erkennt Boxen; Prolix ergänzt die Bedeutung



SGBlur

„Speedy Gonzales“
Blurring Algorithm · Panoramax

Ursprüngliches Repository: cquest/sgblur

Originalprojekt und Namensherkunft

Prolix

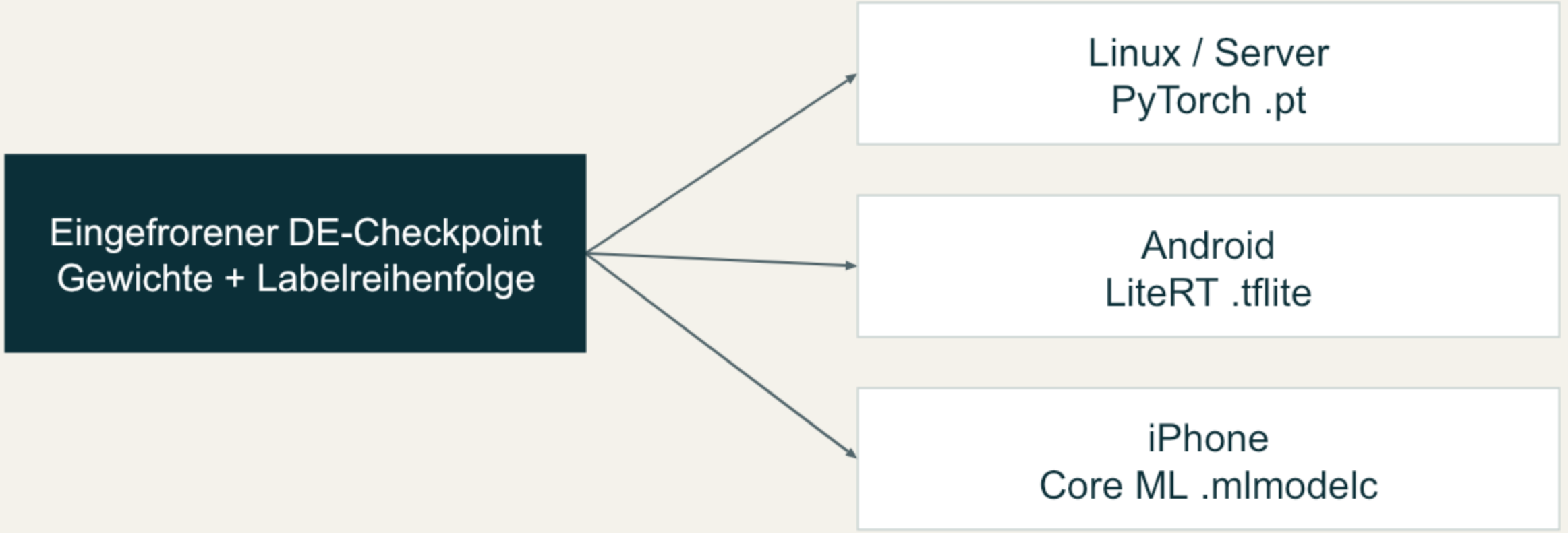
Panoramax-Werkzeug für
Objekterkennung und Klassifikation

Autor: Adrien Pavie · Panoramax-Projekt

Originalprojekt: Panoramax / Prolix

Änderungen am gemeinsamen Detektor: Gesicht und Kennzeichen mitprüfen

Ein Klassifikator-Checkpoint, drei Laufzeitformate



Gemeinsame Gewichte sind die Basis; gleiche Ergebnisse müssen wir nachweisen.

Welche Hardware rechnet die Inferenz?

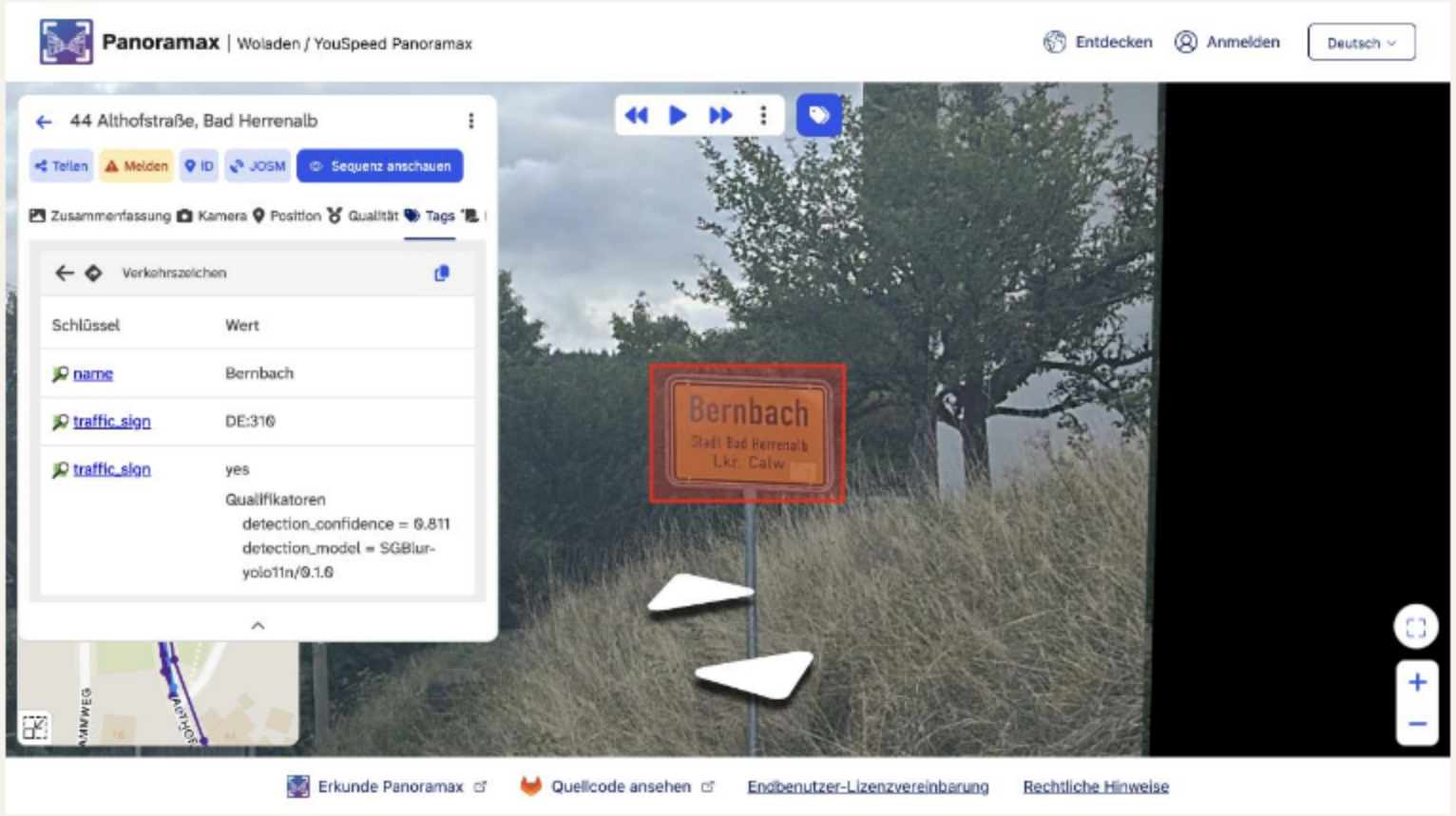
Ziel	Beschleunigung	Was ist belegt?
Linux / KI-Server	NVIDIA CUDA	2 × RTX A5000, je 24 GiB RTX PRO 6000 Blackwell, 96 GiB
Android-Testgerät	LiteRT GPU-Delegate	Mali-G615 MC2; CPU/XNNPACK bei Fehler
iPhone	Core ML mit .all	CPU, Apple GPU und Neural Engine erlaubt

Belegt ≠ fest zugewiesen: die konkrete SGBlur-Host-GPU ist nicht nachgewiesen.

Die Vorverarbeitung gehört zum Modellvertrag

01	**Koordinaten**	Orientierung und Bildgröße müssen zur Annotation passen
02	**Detektor**	RGB · rechtes 1024²-Fenster, volle Höhe
03	**Klassifikator**	Feste Ränder · RGB / 255 · Resize auf 224 × 224
04	**Vergleich**	Labelreihenfolge, Fensterkoordinaten und Scores kontrollieren

Panoramax: Box zeichnen, Bedeutung prüfen



Eine Box pro sichtbarer Tafel

DE:310 = Ortseingang
DE:311 = Ortsausgang

Objekt-Tag: osm|traffic_sign
Speichern und gegenprüfen

Das Label gehört zur Objektbox; ein Bild-Tag allein reicht nicht.

Der Crawler sammelt Beispiele für beide Modelle

Detektor: Vollbilder + Boxen

inbox/detector/<bild-ID>/

Bild + labels.txt + Metadaten

Schilder, Kennzeichen, Gesichter

Prüfen → approved/detector/<bild-ID>/

Klassifikator: Bildausschnitte

Bildausschnitte: inbox/classifier/

DE:310 → city__start

DE:311 → city__end

Prüfen → approved/classifier/<klasse>/

Datenpakete: huggingface.co/datasets/loffenauer/ws26_hspf_imo

Eine feste Struktur für Training, Validierung und Test

```
data/dataset/
  train/<klasse>/    ca. 70 %
  val/<klasse>/      ca. 15 %
  test/<klasse>/     ca. 15 %
  manifest.json
```

Training

Hier lernt das Netz seine Gewichte.

Validierung

Hier wählen Sie den besten Kandidaten.

Test

Einmaliger Abschlusstest nach der Auswahl.

```
python import_sorted_folders.py
```

Beide Sätze verwenden dieselben Gruppen; die Zuordnung zu train / val / test erfolgt automatisch.

Prolix einrichten und seine Rolle verstehen

```
git clone https://gitlab.com/panoramax/clients/prolix.git
cd prolix
git checkout --detach 36bb92bed7d8aba1d87c1fd10fcdf8ec7f06046f
git submodule update --init --recursive
python3.12 -m venv env
source env/bin/activate
python -m pip install -e .
prolix --help
```

Prolix: Inferenz → räumlich zusammenführen → Annotationen

Training und Ground-Truth-Evaluation: PyTorch / Ultralytics

Aktuelles upstream main und der gepinnte Deploymentstand unterscheiden sich.

Datenpakete auf dem KI-Server herunterladen

huggingface.co/datasets/loffenauer/ws26_hspf_imo

Vollständige Lernsammlung: 220 Vollbilder und 334 Bildausschnitte

```
.venv-hf/bin/hf download loffenauer/ws26_hspf_imo \
  --include 'packages/panoramax-studenten-*' \
  --include 'packages/panoramax-mobidata-*' --include LEARNING_COLLECTION.json \
  --include SHA256SUMS.txt --type dataset \
  --revision e74fc1564aae8f74bc4cca0e86baf7cfd612a6c8 --local-dir data/downloads
```

MobiData BW: 57 + 63 Paare; drei Lernarchive insgesamt

Klassen und Boxen prüfen; Testkandidaten getrennt aufbewahren

Vollständige Befehle: praktikum/DATEN_HERUNTERLADEN.md

Nur bei fehlenden Boxen den Detektor nachtrainieren

Vollbilder mit vollständigen Boxen aller drei Klassen prüfen

DE:310 und DE:311 bleiben beim Detektor generisch sign

Import schreibt YOLO-Bilder, Labels und feste Gruppensplits

Gesicht / Kennzeichen im unabhängigen Vergleich mitprüfen

```
yolo detect train model=models/detector-baseline.pt \
  data=data/reviewed/detector/data.yaml \
  epochs=20 imgsz=1280 device=0 seed=20261026
```

SGBlur-YOLO11n: yolo11n_panoramax.pt → models/detector-baseline.pt

Fine-Tuning mit dem Panoramax-Modell

Ein vortrainiertes Modell lernt mit beschrifteten Bildern weiter.
Dabei ändern sich seine Gewichte.

Ausgangsmodell: Panoramax/classify_de_road_signs

YOLO26l-cls · 134 Klassen · Bildausschnitte mit 224 × 224 Pixeln

Gewichte übernehmen, Classification Head auf 136 Klassen erweitern

Mit alten Schildern und geprüften Ortstafeln gemeinsam weiterlernen

Validierung wählt best.pt. Der Abschlusstest folgt danach.

```
python train.py --epochs 20
```

Eine Epoche = ein Durchlauf. Beginnen Sie mit 20 Epochen und prüfen Sie die Validierung.

Precision, Recall und F1 beim Klassifizieren

Erfundenes Rechenbeispiel: 10 Bildausschnitte, nur DE:310 und DE:311

Tatsächlich	Vorhersage DE:310	Vorhersage DE:311
6 × DE:310 Ortseingang	4	2
4 × DE:311 Ortsausgang	1	3

Für DE:310: 4 richtig, 1 falsch gemeldet,
2 als DE:311 eingeordnet

Precision (P)

Wie viele Vorhersagen stimmen?
richtig / Vorhersagen dieser Klasse

Recall (R)

Wie viele echte Beispiele finden wir?
richtig / tatsächliche Beispiele dieser Klasse

F1: gemeinsamer Wert

$2 \times P \times R / (P + R)$

Nur hoch, wenn beide Werte hoch sind.

Klasse	Precision	Recall	F1
DE:310	4/5 = 80 %	4/6 ≈ 67 %	8/11 ≈ 73 %
DE:311	3/5 = 60 %	3/4 = 75 %	6/9 ≈ 67 %

Fortschritt messen – und alte Klassen bewahren

```
python evaluate.py \
  --candidate runs/versuch-01/weights/best.pt
```

Prüfung	Ihre Auswertung nach der Kandidatenwahl
Neue Klassen: Bildausschnitte	DE:310 / DE:311: Support, Precision, Recall und F1
134 Bestandsklassen	Jede Klasse auf denselben Prüfbeispielen vergleichen
Recall / F1 je Klasse	Rückgänge melden; fehlende Abdeckung kennzeichnen

Kandidaten mit Validierung wählen. Erst danach den Test auswerten.
Neue und bestehende Klassen getrennt beurteilen.

Denselben Kandidaten auf allen Zielen prüfen

1	**Einfrieren**	Gewichte-Hash, Datensatz-/Split-Hash, Labelindex und Konfiguration
2	**Konvertieren**	PyTorch → Core ML / LiteRT; getrennte Paketstände dokumentieren
3	**Vergleichen**	Gleiche Bildausschnitte und Vollbilder; Semantik, Boxen, Scores und Laufzeit
4	**Integrieren**	Vorhandene Schnittstellen nutzen; auf beiden Telefonen prüfen

Der DE-Ortstafel-Export ist anzupassen; das vorhandene Fremdländer-CLI akzeptiert kein DE.

Partnerübung: drei Fehler, drei nächste Schritte

A Im Vollbild fehlt die Ortstafelbox. Der manuelle Bildausschnitt wird richtig erkannt.

B Die Box passt. Das alte Modell liefert bad; DE:310 fehlt im Inventar.

C Test mit Bildausschnitten sehr gut. Auf einer neuen Strecke sinkt die Erkennung stark.

Welche Stufe, welche zusätzlichen Daten und welcher Test helfen jeweils?

Ihr Ergebnis ist ein reproduzierbarer Modellversuch

01	**Daten**	Geprüfte Boxen / Bildausschnitte, Herkunft und unabhängige Schildgruppen
02	**Code**	Trainingsumgebung, Seed, Klassenindex und Versionsstände
03	**Beleg**	Baseline ↔ Kandidat; neue Klassen, Altklassen und Fehlbilder
04	**Übertragung**	Checkpoint + Exporte; Linux, Android und iPhone vergleichen

Fragen?

Wie würden Sie jetzt den ersten Versuch für DE:310 und DE:311 aufsetzen?